

SKILL-4

INPUT-

```
GNU nano 8.7.1 skill4.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 50
#define MAX_LEN 100

// Token structure
struct Token {
    char value[MAX_LEN];
};

// Parse tree node
struct Node {
    char value[MAX_LEN];
    struct Node *left;
    struct Node *right;
};

// Check whether character is a delimiter
int isDelimiter(char ch) {
    return ch == ' ' || ch == '\t' ||
           ch == '\n' || ch == '|' ||
           ch == ';';
}

// Tokenize input
int tokenize(char *input, struct Token tokens[]) {
    int count = 0;
    int i = 0;

    while (input[i] != '\0') {

        // Skip whitespace
        while (input[i] != '\0' && isspace(input[i])) {
            i++;
        }
    }
}
```

Help	Write Out	Where Is	Cut	Execute
Exit	Read File	Replace	Paste	Justify

```
}

if (input[i] == '\\0') {
    break;
}

// Handle delimiters
if (input[i] == '|' || input[i] == ';' ) {
    tokens[count].value[0] = input[i];
    tokens[count].value[1] = '\\0';
    count++;
    i++;
    continue;
}

// Store normal token
int j = 0;

while (input[i] != '\\0' &&
        !isspace(input[i]) &&
        input[i] != '|' &&
        input[i] != ';' ) {

    if (j < MAX_LEN - 1) {
        tokens[count].value[j++] = input[i];
    }

    i++;
}

tokens[count].value[j] = '\\0';
count++;

if (count >= MAX_TOKENS) {
    break;
}

| }
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
GNU nano 8.7.1 skill4.c *  
  
    return count;  
}  
  
// Validate token stream  
int validateTokens(struct Token tokens[], int count) {  
    if (count == 0) {  
        printf("Empty command detected.\n");  
        return 0;  
    }  
  
    // Command cannot start with |  
    if (strcmp(tokens[0].value, "|") == 0) {  
        printf("Syntax Error: Command cannot start with '|'.\n");  
        return 0;  
    }  
  
    // Command cannot end with |  
    if (strcmp(tokens[count - 1].value, "|") == 0) {  
        printf("Syntax Error: Command cannot end with '|'.\n");  
        return 0;  
    }  
  
    // Two pipes cannot occur together  
    for (int i = 0; i < count - 1; i++) {  
        if (strcmp(tokens[i].value, "|") == 0 &&  
            strcmp(tokens[i + 1].value, "|") == 0) {  
            printf("Syntax Error: Two '|' symbols cannot be consecutive.\n");  
            return 0;  
        }  
    }  
  
    return 1;  
}  
|  
  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

GNU nano 8.7.1 skill4.c *

```
// Create parse tree
struct Node *createNode(char *value) {

    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        exit(1);
    }

    strcpy(newNode->value, value);
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

// Display parse tree
void displayTree(struct Node *root, int level) {

    if (root == NULL) {
        return;
    }

    for (int i = 0; i < level; i++) {
        printf(" ");
    }

    printf("|-- %s\n", root->value);

    displayTree(root->left, level + 1);
    displayTree(root->right, level + 1);
}

// Build a simple execution structure
```

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^_ Replace

^K Cut

^U Paste

^T Execute

^J Justify

```
GNU nano 8.7.1 skill4.c *
// Build a simple execution structure
struct Node *buildTree(struct Token tokens[], int count) {

    struct Node *root = createNode(tokens[0].value);

    struct Node *current = root;

    for (int i = 1; i < count; i++) {

        if (strcmp(tokens[i].value, "|") == 0) {

            current->right = createNode("|");
            current = current->right;

        } else {

            if (current->left == NULL) {
                current->left = createNode(tokens[i].value);
            } else {
                current->right = createNode(tokens[i].value);
                current = current->right;
            }

        }

    }

    return root;
}

// Free parse tree memory
void freeTree(struct Node *root) {

    if (root == NULL) {
        return;
    }

    freeTree(root->left);
    freeTree(root->right);
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

GNU nano 8.7.1 skill4.c *

```
    free(root);
}

int main() {

    char input[500];

    struct Token tokens[MAX_TOKENS];

    printf("=====\n");
    printf("    TOKENIZER AND PARSER DEMO\n");
    printf("=====\n");

    printf("\nEnter a command: ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Input error.\n");
        return 1;
    }

    // Remove newline
    input[strcspn(input, "\n")] = '\0';

    // Tokenization
    int count = tokenize(input, tokens);

    printf("\n--- Tokens ---\n");

    if (count == 0) {
        printf("No tokens found.\n");
        return 0;
    }

    for (int i = 0; i < count; i++) {
        printf("Token %d: [%s]\n", i + 1, tokens[i].value);
    }
}
```

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^_ Replace

^K Cut

^U Paste

^T Execute

^J Justify

```
GNU nano 8.7.1 skill4.c *

if (count == 0) {
    printf("No tokens found.\n");
    return 0;
}

for (int i = 0; i < count; i++) {
    printf("Token %d: [%s]\n", i + 1, tokens[i].value);
}

// Validate tokens
printf("\n--- Token Validation ---\n");

if (!validateTokens(tokens, count)) {
    printf("Invalid token stream.\n");
    return 1;
}

printf("Token stream is valid.\n");

// Parser
printf("\n--- Parse Tree ---\n");

struct Node *root = buildTree(tokens, count);

displayTree(root, 0);

printf("\nExecution structure created successfully.\n");

// Release memory
freeTree(root);

printf("Memory released successfully.\n");

return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

OUTPUT-

```
bhagya_sri@DESKTOP-CDJB x + v
bhagya_sri@DESKTOP-CDJBPHA:~$ pwd
/home/bhagya_sri
bhagya_sri@DESKTOP-CDJBPHA:~$ mkdir OS-SKILL-4
bhagya_sri@DESKTOP-CDJBPHA:~$ cd OS-SKILL-4
bhagya_sri@DESKTOP-CDJBPHA:~/OS-SKILL-4$ nano skill4.c
bhagya_sri@DESKTOP-CDJBPHA:~/OS-SKILL-4$ gcc -Wall -Wextra -g skill4.c -o skill4
bhagya_sri@DESKTOP-CDJBPHA:~/OS-SKILL-4$ ./skill4
=====
          TOKENIZER AND PARSER DEMO
=====

Enter a command: Hello world

--- Tokens ---
Token 1: [Hello]
Token 2: [world]

--- Token Validation ---
Token stream is valid.

--- Parse Tree ---
|-- Hello
   |-- world

Execution structure created successfully.
Memory released successfully.
bhagya_sri@DESKTOP-CDJBPHA:~/OS-SKILL-4$ valgrind --leak-check=full ./skill4
==1559== Memcheck, a memory error detector
==1559== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1559== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1559== Command: ./skill4
==1559==
=====
          TOKENIZER AND PARSER DEMO
=====

Enter a command: ls -1 | grep txt

--- Tokens ---
Token 1: [ls]
```

```

bhagya_sri@DESKTOP-CDJB  x  +  v  -  □  ×
bhagya_sri@DESKTOP-CDJBPHA:~/OS-SKILL-4$ valgrind --leak-check=full ./skill4
==1559== Memcheck, a memory error detector
==1559== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1559== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1559== Command: ./skill4
==1559==
=====
                TOKENIZER AND PARSER DEMO
=====

Enter a command: ls -l | grep txt

--- Tokens ---
Token 1: [ls]
Token 2: [-l]
Token 3: [|]
Token 4: [grep]
Token 5: [txt]

--- Token Validation ---
Token stream is valid.

--- Parse Tree ---
|-- ls
|  |-- -l
|  |-- |
|     |-- grep
|     |-- txt

Execution structure created successfully.
Memory released successfully.
==1559==
==1559== HEAP SUMMARY:
==1559==      in use at exit: 0 bytes in 0 blocks
==1559==    total heap usage: 7 allocs, 7 frees, 2,648 bytes allocated
==1559==
==1559== All heap blocks were freed -- no leaks are possible
==1559==
==1559== For lists of detected and suppressed errors, rerun with: -s
==1559== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
bhagya_sri@DESKTOP-CDJBPHA:~/OS-SKILL-4$ |
```